L38 — AVL Trees: Rotations and Balancing

Unit: 3 | **Chapter:** Trees | **BT Level:** BT6 | **CO:** CO2, CO4

Learning Objectives

- Define balance factor and AVL tree property
- Identify the four imbalance cases (LL, RR, LR, RL)
- Perform rotations to restore balance after insertion
- Implement AVL insertion with automatic rebalancing

---

1. Motivation

Problem with BST: Inserting elements in sorted order produces a skewed tree with O(n) height → O(n) operations.

AVL Tree (Adelson-Velsky and Landis, 1962) — a self-balancing BST that maintains **height balance** by performing rotations whenever the balance factor of any node exceeds ± 1.

Guarantee: Height of AVL tree $\leq 1.44 \times \log_2(n)$ → all operations O(log n).

---

2. Balance Factor

Balance Factor (BF) of a node = height(left subtree) - height(right subtree)

- BF = -1, 0, or +1: **Balanced**
- BF = +2: **Left heavy** — left subtree too tall
- BF = -2: **Right heavy** — right subtree too tall

```
def height(node):
    return node.height if node else -1

def balance_factor(node):
    return height(node.left) - height(node.right) if node else 0
```

---

3. AVL Node

```
class AVLNode:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None
        self.height = 0   # Leaf height = 0

def update_height(node):
```

```
    node.height = 1 + max(height(node.left), height(node.right))
```

---

4. The Four Imbalance Cases and Rotations

Case 1: Left-Left (LL) — Right Rotation

Scenario: New node inserted in the **left subtree of left child**.
Fix: Single right rotation.

```
Before (BF=+2):    After right rotation:
      z                 y
     / \              /   \
    y   T4           x     z
   / \              / \   / \
  x   T3          T1 T2 T3 T4
 / \
T1 T2
```

```
def rotate_right(z):
    y = z.left
    T3 = y.right

    y.right = z
    z.left = T3

    update_height(z)
    update_height(y)
    return y   # New root of this subtree
```

Case 2: Right-Right (RR) — Left Rotation

Scenario: New node inserted in the **right subtree of right child**.
Fix: Single left rotation.

```
Before (BF=-2):    After left rotation:
    z                   y
   / \                /   \
  T1   y             z     x
      / \           / \   / \
     T2  x         T1 T2 T3 T4
        / \
       T3 T4
```

```
def rotate_left(z):
    y = z.right
    T2 = y.left

    y.left = z
    z.right = T2

    update_height(z)
```

```
    update_height(y)
    return y
```

Case 3: Left-Right (LR) — Double Rotation

Scenario: New node inserted in the **right subtree of left child**.
Fix: Left rotate at left child, then right rotate at z.

```
Before:    After left-rotate(y):    After right-rotate(z):
    z              z                       x
   / \            / \                     /   \
  y   T4         x   T4                  y     z
 / \            / \                     / \   / \
T1  x          y   T3                  T1 T2 T3 T4
   / \        / \
  T2 T3      T1 T2
```

```
def rotate_left_right(z):
    z.left = rotate_left(z.left)
    return rotate_right(z)
```

Case 4: Right-Left (RL) — Double Rotation

Scenario: New node inserted in the **left subtree of right child**.
Fix: Right rotate at right child, then left rotate at z.

```
def rotate_right_left(z):
    z.right = rotate_right(z.right)
    return rotate_left(z)
```

---

5. Rebalance Helper

```
def rebalance(node):
    update_height(node)
    bf = balance_factor(node)

    # Left heavy
    if bf > 1:
        if balance_factor(node.left) >= 0:
            return rotate_right(node)          # LL case
        else:
            return rotate_left_right(node)     # LR case

    # Right heavy
    if bf < -1:
        if balance_factor(node.right) <= 0:
            return rotate_left(node)           # RR case
        else:
            return rotate_right_left(node)     # RL case
```

```
    return node   # Already balanced
```

---

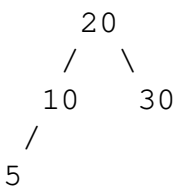
6. AVL Insert

```
def avl_insert(root, data):
    """Insert data into AVL tree rooted at root. Returns new root."""
    # Standard BST insert
    if root is None:
        return AVLNode(data)

    if data < root.data:
        root.left = avl_insert(root.left, data)
    elif data > root.data:
        root.right = avl_insert(root.right, data)
    else:
        return root   # Duplicate

    return rebalance(root)
```

---

7. Insertion Trace: Insert 30 into tree

Initial AVL tree:

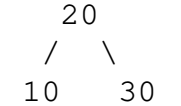


Insert 5 into fresh tree: 10, 20, 30, 5, 25

Let's trace inserting into an empty tree:

Insert	Tree	BF at root	Action
10	`10`	0	Balanced
20	`10→20`	-1	Balanced
30	`10→20→30`	-2 at 10	**RR case** → Left rotate at 10

After left rotate:



Insert	Node added	BF violation	Fix
5	5 as left of 10	BF = 1 at 20 → ok	None
25	25 as left of 30	BF = -1 at 20 → ok	None

Final tree:

```
     20
    /  \
   10   30
  /   /
 5   25
```

---

8. AVL Delete (Outline)

Deletion follows BST deletion, then rebalances from the deleted node's parent back up to the root (potentially requiring O(log n) rotations).

```
def avl_delete(root, data):
    if root is None:
        return None

    if data < root.data:
        root.left = avl_delete(root.left, data)
    elif data > root.data:
        root.right = avl_delete(root.right, data)
    else:
        # Found node to delete
        if root.left is None:
            return root.right
        if root.right is None:
            return root.left
        # Two children: replace with in-order successor
        successor = root.right
        while successor.left:
            successor = successor.left
        root.data = successor.data
        root.right = avl_delete(root.right, successor.data)

    return rebalance(root)
```

---

9. Complexity Analysis

Operation	AVL Tree	Unbalanced BST
Search	O(log n)	O(n) worst
Insert	O(log n)	O(n) worst
Delete	O(log n)	O(n) worst
Space	O(n)	O(n)
Height	$\leq 1.44 \log_2 n$	Up to n-1

At most 1 rotation needed per insert; at most O(log n) rotations for delete.

Summary

- AVL tree: BST with balance factor $\in \{-1, 0, +1\}$ at every node.
- Four imbalance cases: **LL** (right rotate), **RR** (left rotate), **LR** (left then right), **RL** (right then left).
- Insert triggers at most **1 double rotation** to fix imbalance.
- All operations guaranteed **O(log n)** due to height bound.

References

- Cormen et al., *Introduction to Algorithms*, Ch. 13 (MIT Press, 2022)
- Skiena, *The Algorithm Design Manual*, Ch. 3.4 (Springer, 2020)